Lista de Exercícios — Lógica de Programação com ESP32/Arduino

Oct 9, 2026 · @Thiago

Antes de começar

Esta lista tem 11 exercícios, do mais simples ao mais completo, para fixar o conteúdo visto em aula e treinar lógica de programação: **if/else, for, switch case, operador ternário, controle de tempo sem delay com millis(), LCD, push button e LED**, usando as classes `Led` e `Botao` que construímos juntos.

Como usar esta lista

1. **Estude primeiro os fluxogramas postados no Classroom**: o do botão clássico e o do botão com cláusulas de guarda. O método `Botao::atualizar()` segue exatamente o fluxograma com cláusulas de guarda: cada `if (...) return;` é uma "porta" que encerra a função cedo quando ainda não é hora de agir. Vários exercícios usam essa mesma ideia.
2. Monte uma única vez o circuito padrão da próxima seção. Todos os exercícios usam o mesmo circuito.
3. Leia a explicação, o enunciado e a dica de cada exercício e tente resolver sozinho antes de olhar o gabarito.
4. Só depois confira o **Gabarito comentado**. Ele traz o código e a explicação da lógica. Se a sua solução for diferente e funcionar, ótimo: existem vários caminhos certos.
5. As dúvidas sobre os exercícios podem ser trazidas para a aula.

Regras do jogo

- **Proibido usar `delay()`**. Todo controle de tempo deve ser feito com `millis()`.
- Não é preciso nenhum algoritmo pronto. Tudo se resolve com variáveis, condições, repetições e as classes `Led` e `Botao`.
- A lógica é a mesma para a ESP32. Usamos o Arduino Uno apenas porque é ele que o Tinkercad simula.

Montagem padrão (use em todos os exercícios)

No Tinkercad, monte uma vez o circuito abaixo com um **Arduino Uno R3**, um **LCD 16 x 2 (I2C)**, **3 LEDs** e **2 botões**. Na ESP32 a lógica do código não muda: trocam apenas os pinos e as três linhas de configuração do LCD. O DHT22 entra só no exercício 11.

Componente	Nome no código	Arduino Uno (Tinkercad)	ESP32
LED vermelho (+ resistor 220 Ω)	`ledVermelho`	10	GPIO 25
LED amarelo (+ resistor 220 Ω)	`ledAmarelo`	9	GPIO 26
LED verde (+ resistor 220 Ω)	`ledVerde`	8	GPIO 27
Botão A (outro terminal no GND)	`botaoA`	7	GPIO 32
Botão B (outro terminal no GND)	`botaoB`	6	GPIO 33
LCD I2C: SDA	`lcd`	A4	GPIO 21
LCD I2C: SCL	`lcd`	A5	GPIO 22
LCD I2C: VCC e GND	—	5V e GND	5V (VIN) e GND
DHT22: DATA (só exercício 11)	`dht`	—	GPIO 4

Por que LCD I2C? O LCD 16x2 comum usa 6 pinos de dados. Com o módulo I2C, ele usa só 2 (SDA e SCL), além da alimentação. Cada dispositivo I2C tem um **endereço**, e é por ele que o programa encontra o LCD.

Botões: não use resistor externo. A classe `Botao` liga o resistor interno com `INPUT_PULLUP`. Por isso o pino lê `HIGH` com o botão solto e `LOW` com o botão pressionado.

O que muda no código entre o Tinkercad e a ESP32: só a biblioteca, a criação do objeto `lcd` e duas linhas do `setup()`. Os comandos `lcd.clear()`, `lcd.setCursor()` e `lcd.print()` são iguais nas duas.

Parte do código	Tinkercad (Arduino Uno)	ESP32 (placa real ou Wokwi)
Biblioteca	`#include <Adafruit_LiquidCrystal.h>`	`#include <LiquidCrystal_I2C.h>`
Objeto	`Adafruit_LiquidCrystal lcd(0);`	`LiquidCrystal_I2C lcd(0x27, 16, 2);`

Parte do código	Tinkercad (Arduino Uno)	ESP32 (placa real ou Wokwi)
No `setup()`	`lcd.begin(16, 2);` e `lcd.setBacklight(1);`	`lcd.init();` e `lcd.backlight();`

Nos módulos reais, o endereço mais comum é `0x27`; alguns usam `0x3F`. Se a tela acender e não mostrar nada, troque o endereço.

Observações para a ESP32

- Não use os GPIOs 6 a 11 (ligados à memória flash da placa).
- Não use os GPIOs 34 a 39 para botões: eles não têm resistor de pull-up interno.
- Para simular a ESP32, use o Wokwi, simulador gratuito no navegador. Ele tem ESP32, LCD 16x2 I2C e DHT22. O Tinkercad não tem o DHT22, por isso o exercício 11 é feito no Wokwi. As bibliotecas são adicionadas pela aba **Library Manager**.

Estrutura do código: o editor do Tinkercad aceita apenas um arquivo. Por isso, em todos os exercícios, cole no topo do código o bloco do **Apêndice — Classes Led e Botao** (as mesmas classes vistas em aula, reunidas em um arquivo só). No PlatformIO com a ESP32, use os arquivos `led.h/led.cpp` e `botao.h/botao.cpp` normalmente.

Sensor DHT22: o que é e como usar

O DHT22 (também vendido como AM2302) é um sensor **digital** que mede a **temperatura** e a **umidade relativa do ar** e envia os valores prontos para o microcontrolador por um único pino de dados.

Característica	DHT22
Alimentação	3 a 5 V (funciona direto no 3,3 V da ESP32)
Temperatura	−40 a 80 °C, precisão de ±0,5 °C
Umidade relativa	0 a 100 %, precisão de 2 a 5 %
Velocidade máxima	1 leitura a cada 2 segundos
Comunicação	1 fio de dados, com protocolo próprio (não é analógico nem I2C)

Fonte das especificações: Adafruit — DHT11, DHT22 and AM2302 Sensors.

Ligação (sensor de 4 pinos, da esquerda para a direita com a grade virada para você): 1 = VCC (3V3 da ESP32), 2 = DATA (GPIO 4), 3 = não usado, 4 = GND. Coloque um resistor de 10 kΩ entre DATA e VCC (pull-up). Os módulos de 3 pinos já vêm com esse resistor na plaquinha.

Biblioteca: "DHT sensor library", da Adafruit (ela pede também a "Adafruit Unified Sensor"). No Wokwi, adicione as duas pelo Library Manager, junto com a "LiquidCrystal I2C".

Como usar no código

```
#include <DHT.h>

DHT dht(4, DHT22);          // pino de dados e modelo do sensor

void setup()
{
    dht.begin();            // prepara o sensor
}

void loop()
{
    float temperatura = dht.readTemperature();   // em °C
    float umidade = dht.readHumidity();          // em %

    if(isnan(temperatura)) return;   // leitura falhou: não use o valor
}
```

- `readTemperature()` e `readHumidity()` devolvem `float`. Quando a leitura falha (fio solto, sensor ausente), devolvem **NaN** ("não é um número"), que é testado com `isnan()`.
- O sensor é lento: não adianta ler mais rápido que a cada 2 s. Por isso a leitura deve ser controlada com `millis()`, nunca com `delay()`.
- No Wokwi, clique no DHT22 durante a simulação para mudar a temperatura e a umidade com um controle deslizante. É assim que você testa o exercício 11.

Exercícios

Os exercícios estão em ordem de dificuldade. Do 2 em diante, use as classes `Led` e `Botao`, e lembre que `Led::atualizar()` e `Botao::atualizar()` precisam ser chamados em **toda volta do loop()**.

Exercício 1 — Dois LEDs, dois ritmos

Conteúdos: millis(), if.

Explicação: com `delay()`, o programa fica parado e não consegue cuidar de duas coisas ao mesmo tempo. Com `millis()`, o programa só pergunta "já passou o tempo?" e segue em frente. Neste exercício você faz isso **sem as classes**, para entender o que a classe `Led` faz por dentro.

O que fazer:

- O LED vermelho deve trocar de estado a cada 500 ms.
- O LED verde deve trocar de estado a cada 1300 ms.
- Os dois devem piscar ao mesmo tempo, cada um no seu ritmo, sem `delay()` e sem a classe `Led`.

Dica: cada LED precisa da sua própria variável de "última troca" e do seu próprio estado. Compare com o método `Led::atualizar()`.

Exercício 2 — Contador de cliques: par ou ímpar?

Conteúdos: push button (classe Botao), LCD, if, operador ternário.

Explicação: o método `pressionou()` fica verdadeiro em **uma única volta** do loop, logo depois que o botão é apertado. É isso que permite contar um clique por vez, sem contar o mesmo clique várias vezes.

O que fazer:

- Botão A soma 1 ao contador. Botão B zera o contador.
- Linha 1 do LCD: `Cliques: N`.
- Linha 2 do LCD: `Numero PAR` ou `Numero IMPAR`, escolhido com o **operador ternário**.
- LED verde aceso quando o número for par; LED vermelho aceso quando for ímpar.
- Atualize o LCD **somente quando o valor mudar**.

Dica: o resto da divisão (`%`) por 2 diz se o número é par. Use uma variável `bool` do tipo "precisa atualizar a tela" e uma cláusula de guarda.

Exercício 3 — Seletor de modos

Conteúdos: switch case, ternário, métodos da classe Led.

Explicação: quando uma variável pode assumir vários valores e cada um tem um comportamento, o `switch case` deixa o código mais organizado que uma sequência de `if/else`.

O que fazer: o botão A avança o modo (0 → 1 → 2 → 3 → 0...) e o botão B volta direto ao modo 0. O LCD mostra o número e o nome do modo.

Modo	Nome no LCD	Comportamento dos LEDs
0	Desligado	Todos apagados
1	Todos acesos	Os 3 LEDs acesos
2	Todos piscando	Os 3 LEDs piscando a cada 500 ms
3	Alerta	Só o vermelho piscando rápido (150 ms)

Dica: antes de aplicar um modo novo, "limpe" o anterior com `desativarPiscar()` em todos os LEDs. Para voltar ao modo 0 depois do 3, experimente o ternário.

Exercício 4 — Barra de nível

Conteúdos: for, if/else, cláusulas de guarda, LCD.

Explicação: o `for` repete uma ação um número conhecido de vezes. Aqui ele desenha uma barra no LCD, um caractere por vez.

O que fazer:

- O nível vai de 0 a 10. Botão A sobe 1, botão B desce 1. O nível nunca passa de 10 nem fica abaixo de 0.
- Linha 1 do LCD: `Nivel: 70%` (nível × 10).
- Linha 2 do LCD: uma barra como `[#######---]`, com um `#` por nível e `-` no restante, desenhada com `for`.
- LEDs: verde aceso a partir do nível 1; amarelo aceso a partir do 4; vermelho aceso a partir do 8 e **piscando** no nível 10.

Dica: use cláusulas de guarda nas funções de subir e descer (`if (nivel == 10) return;`). Dois `for` resolvem a barra.

Exercício 5 — Cronômetro

Conteúdos: millis(), if/else, ternário, LCD.

Explicação: um cronômetro que pode ser pausado precisa guardar o tempo já contado antes da pausa. O tempo total é "o que já foi guardado" mais "o que passou desde que voltou a rodar".

O que fazer:

- Botão A inicia e pausa. Botão B zera, mas **somente se estiver pausado**.

- Linha 1 do LCD: `Rodando` ou `Pausado`. Linha 2: o tempo no formato `mm:ss.d` (minutos, segundos e décimos).
- LED verde aceso enquanto roda. LED amarelo aceso quando estiver pausado com tempo maior que zero.
- Atualize o LCD a cada 100 ms, usando `millis()`.

Dica: para mostrar `05` em vez de `5`, use um `if` que imprime o `0` antes quando o número for menor que 10. Evite `lcd.clear()` a cada atualização: a tela fica piscando.

Exercício 6 — Teste de reflexo

Conteúdos: switch case (máquina de estados), millis(), if/else, LCD.

Explicação: um jogo tem etapas (esperando, contando, mostrando resultado). Cada etapa é um **estado**, guardado em uma variável. O `switch` executa só o código do estado atual, e cada estado decide quando passar para o próximo.

O que fazer:

1. Tela inicial: `Teste de reflexo` / `A: comecar`.
2. Ao apertar A, sorteie uma espera entre 2 e 5 segundos com `random()` e mostre `Prepare-se...`.
3. Passada a espera, acenda o LED verde e comece a medir o tempo.
4. Quando o jogador apertar B, mostre o tempo de reação em ms e uma avaliação: menos de 250 ms `Excelente!`, menos de 400 ms `Bom!`, senão `Treine mais!`.
5. Se apertar B **antes** do LED acender, mostre `Queimou largada!` com o LED vermelho piscando.
6. Botão A volta para a tela inicial.

Dica: use constantes para os estados (`const uint8_t ESTADO_INICIO = 0;` etc.). Chame `randomSeed(millis())` no momento do clique para o sorteio mudar a cada partida.

Exercício 7 — Semáforo com botão de pedestre

Conteúdos: switch case, millis(), if/else, ternário, LCD.

Explicação: é outra máquina de estados, agora com tempos fixos e um evento externo (o pedestre) que pode encurtar um dos estados.

O que fazer:

- Ciclo: verde 5 s → amarelo 2 s → vermelho 5 s → verde...
- LCD linha 1: nome do estado (`VERDE: siga`, `AMARELO: atencao`, `VERMELHO: pare`). Linha 2: `Restam: Xs`, atualizada a cada 250 ms.

- Botão A é o botão do pedestre: se apertado durante o verde, o verde termina assim que tiver completado pelo menos 2 s. Mostre `PED` no LCD enquanto o pedido estiver ativo.

Dica: crie uma função `mudarEstado(novoEstado)` que guarda o novo estado, anota o `millis()` e acerta os LEDs. Assim cada `case` só precisa decidir **quando** mudar.

Exercício 8 — Clique curto ou clique longo?

Conteúdos: pressionou() e soltou(), millis(), cláusulas de guarda, ternário.

Explicação: medindo o tempo entre o `pressionou()` e o `soltou()`, um único botão passa a ter duas funções. É assim que muitos aparelhos funcionam (por exemplo, segurar para ligar).

O que fazer:

- Clique curto (menos de 800 ms) no botão A: alterna o LED amarelo.
- Clique longo (800 ms ou mais): liga ou desliga o pisca do LED vermelho (300 ms).
- O LED verde fica aceso enquanto o botão está pressionado.
- O LCD mostra o tipo de clique e quanto tempo ele durou em ms.

Dica: guarde o `millis()` no `pressionou()`; calcule a duração no `soltou()`. Monte a função com cláusulas de guarda, como no fluxograma do Classroom.

Exercício 9 — Crie a sua classe: Contador

Conteúdos: POO (atributos privados, construtor, métodos, getters), cláusulas de guarda, if/else, ternário.

Explicação: agora é a sua vez de criar uma classe, seguindo o mesmo modelo das classes `Led` e `Botao`: atributos em `private`, métodos em `public` e construtor com lista de inicialização.

O que fazer:

1. Crie a classe `Contador` com os atributos privados `_valor` (começa em 0) e `_limite` (recebido no construtor).
2. Métodos: `incrementar()` (não passa do limite), `decrementar()` (não fica abaixo de 0), `zerar()`, `getValor()`, `getLimite()`, `atingiuLimite()` e `estaZerado()`.
3. Use a classe para controlar um estacionamento de 5 vagas: botão A = entrou um carro, botão B = saiu um carro.
4. LCD: `Ocupadas: N/5` e, na linha 2, `LOTADO` ou `Ha vagas` (ternário).
5. LEDs: verde com o estacionamento vazio, amarelo com vagas ocupadas, vermelho piscando quando lotado.

Dica: os limites ficam **dentro** da classe, com cláusulas de guarda. Assim o `loop()` não precisa se preocupar com eles.

Exercício 10 — Desafio integrador: temporizador de cozinha

Conteúdos: todos (switch, if/else, for, ternário, millis(), LCD, botões, LEDs).

Explicação: este exercício junta tudo em um aparelho real. Planeje primeiro os estados e o que cada botão faz em cada um, de preferência desenhando um fluxograma.

O que fazer:

Estado	Tela	LEDs	Botão A	Botão B
Ajuste	`Ajuste: mm:ss` e `A:+10s B:inicia`	Amarelo aceso	Soma 10 s (máximo 5 min; passou disso, volta a 10 s)	Inicia a contagem, se o tempo for maior que zero
Contando	`Faltam: mm:ss` e barra de progresso com 16 caracteres	Verde piscando a cada 1 s	—	Cancela e volta ao ajuste
Alarme	`TEMPO ESGOTADO!`	Os 3 piscando a cada 200 ms	Volta ao ajuste	Volta ao ajuste

Dica: a barra de progresso é um `for` de 0 a 15 que imprime `#` ou `-` com um ternário, conforme o tempo que já passou. Atualize a tela a cada 200 ms.

Exercício 11 — Monitor de temperatura com DHT22 (ESP32 no Wokwi)

Conteúdos: millis(), cláusulas de guarda, if, ternário, LCD I2C, LEDs, sensor DHT22.

Explicação: um sensor não precisa ser lido a toda volta do `loop()`, e a tela não precisa ser reescrita se nada mudou. Aqui você controla **quando ler** (a cada 5 s, com `millis()`) e **quando mostrar** (só se a temperatura variou). Leia antes a seção **Sensor DHT22: o que é e como usar**.

O que fazer (no Wokwi, com ESP32, LCD 16x2 I2C, DHT22 no GPIO 4, LED vermelho no GPIO 25 e LED verde no GPIO 27):

1. Ao ligar, mostre `Lendo sensor...`.
2. Leia a temperatura do DHT22 **a cada 5 segundos**, controlando o tempo com `millis()`.
3. Atualize o LCD **somente se a temperatura mudou** em relação ao último valor mostrado, considerando uma casa decimal (25,3 °C para 25,4 °C conta como mudança).

4. Linha 1 do LCD: `Temp: 25.3°C`. Linha 2: `Normal` ou, a partir de 30,0 °C, `ALERTA: quente!` (use o ternário).
5. LEDs: verde aceso quando normal; vermelho piscando (300 ms) em alerta.
6. Se a leitura falhar, mostre `Erro no sensor`.

Dica: o `loop()` fica com três cláusulas de guarda em sequência: "ainda não deu 5 s? `return`", "a leitura falhou? `return`", "o valor não mudou? `return`". Compare a temperatura como número inteiro em décimos de grau (`round(temperatura * 10)`) em vez de comparar `float` com `==`. O símbolo de grau no LCD é `lcd.print((char)223);`.

Gabarito comentado

Tente resolver antes de ler. Do exercício 2 em diante, a linha `// >>> Cole aqui o bloco do Apêndice <<<` indica onde colar as classes `Led` e `Botao`. Todos os códigos seguem a mesma ordem dentro do `loop()`: **1) ler as entradas, 2) decidir, 3) atualizar as saídas, 4) atualizar a tela**.

Gabarito 1 — Dois LEDs, dois ritmos

```
// Exercício 1 — Dois LEDs piscando em ritmos diferentes, sem delay()

const uint8_t PINO_VERMELHO = 10;
const uint8_t PINO_VERDE = 8;

const uint32_t INTERVALO_VERMELHO_ms = 500;
const uint32_t INTERVALO_VERDE_ms = 1300;

uint32_t ultimaTrocaVermelho_ms = 0;
uint32_t ultimaTrocaVerde_ms = 0;

bool estadoVermelho = LOW;
bool estadoVerde = LOW;

void setup()
{
    pinMode(PINO_VERMELHO, OUTPUT);
    pinMode(PINO_VERDE, OUTPUT);
}

void loop()
{
    const uint32_t agora_ms = millis();

    // Já passou o tempo do LED vermelho?
```

```
    if(agora_ms - ultimaTrocaVermelho_ms >= INTERVALO_VERMELHO_ms)
    {
        ultimaTrocaVermelho_ms = agora_ms;
        estadoVermelho = !estadoVermelho;
        digitalWrite(PINO_VERMELHO, estadoVermelho);
    }

    // Já passou o tempo do LED verde?
    if(agora_ms - ultimaTrocaVerde_ms >= INTERVALO_VERDE_ms)
    {
        ultimaTrocaVerde_ms = agora_ms;
        estadoVerde = !estadoVerde;
        digitalWrite(PINO_VERDE, estadoVerde);
    }
}
```

Explicação

- `millis()` devolve há quantos milissegundos a placa está ligada. A conta `agora_ms - ultimaTroca_ms` diz quanto tempo passou desde a última troca.
- Quando esse tempo chega ao intervalo, o LED troca de estado (`!estado` inverte o valor) e a "última troca" passa a ser agora.
- Como nenhum `if` trava o programa, o `loop()` roda milhares de vezes por segundo e cuida dos dois LEDs "ao mesmo tempo".
- É exatamente o que acontece dentro de `Led::atualizar()` quando o LED está piscando. A classe só esconde esses detalhes.

Gabarito 2 — Contador de cliques: par ou ímpar?

```
// Exercício 2 — Contador de cliques com par/ímpar

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

uint16_t contador = 0;
bool precisaAtualizarTela = true;
```

```
void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    botaoB.iniciar();
}

void loop()
{
    // 1. Ler as entradas
    botaoA.atualizar();
    botaoB.atualizar();

    // 2. Decidir
    if(botaoA.pressionou())
    {
        contador++;
        precisaAtualizarTela = true;
    }

    if(botaoB.pressionou())
    {
        contador = 0;
        precisaAtualizarTela = true;
    }

    const bool ehPar = (contador % 2 == 0);

    // 3. Atualizar as saídas
    ledVerde.setEstadoLed(ehPar);
    ledVermelho.setEstadoLed(!ehPar);
    ledVerde.atualizar();
    ledVermelho.atualizar();

    // 4. Tela: cláusula de guarda
    if(!precisaAtualizarTela) return;

    precisaAtualizarTela = false;

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Cliques: ");
    lcd.print(contador);
```

```
    lcd.setCursor(0, 1);
    lcd.print(ehPar ? "Numero PAR" : "Numero IMPAR");
}
```

Explicação

- `pressionou()` só é verdadeiro na volta do loop em que o clique foi confirmado (depois do debounce). Por isso cada clique soma exatamente 1.
- `contador % 2` é o resto da divisão por 2: resto 0 significa par. O resultado vai para a variável `ehPar`, usada nos LEDs e na tela.
- `setEstadoLed(ehPar)` aproveita o próprio valor `true/false` para acender ou apagar o LED, sem precisar de `if`.
- O ternário `ehPar ? "Numero PAR" : "Numero IMPAR"` escolhe o texto em uma linha só: se a condição for verdadeira, usa o primeiro valor; se for falsa, o segundo.
- A cláusula de guarda `if(!precisaAtualizarTela) return;` impede que o LCD seja reescrito a toda volta. Os LEDs ficam **antes** dela, porque precisam ser atualizados sempre.

Gabarito 3 — Seletor de modos

```
// Exercício 3 — Seletor de modos com switch case

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

const uint8_t TOTAL_MODOS = 4;
uint8_t modo = 0;

void aplicarModo()
{
    // Começa sempre do zero: para de piscar e apaga tudo
    ledVermelho.desativarPiscar();
    ledAmarelo.desativarPiscar();
    ledVerde.desativarPiscar();
```

```
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Modo ");
    lcd.print(modo);
    lcd.setCursor(0, 1);

    switch(modo)
    {
        case 0:
            lcd.print("Desligado");
            break;

        case 1:
            ledVermelho.ligar();
            ledAmarelo.ligar();
            ledVerde.ligar();
            lcd.print("Todos acesos");
            break;

        case 2:
            ledVermelho.ativarPiscar(500);
            ledAmarelo.ativarPiscar(500);
            ledVerde.ativarPiscar(500);
            lcd.print("Todos piscando");
            break;

        case 3:
            ledVermelho.ativarPiscar(150);
            lcd.print("Alerta");
            break;

        default:
            lcd.print("Modo invalido");
            break;
    }
}

void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
```

```
    botaoB.iniciar();
    aplicarModo();
}

void loop()
{
    botaoA.atualizar();
    botaoB.atualizar();

    if(botaoA.pressionou())
    {
        modo = (modo == TOTAL_MODOS - 1) ? 0 : modo + 1;
        aplicarModo();
    }

    if(botaoB.pressionou())
    {
        modo = 0;
        aplicarModo();
    }

    ledVermelho.atualizar();
    ledAmarelo.atualizar();
    ledVerde.atualizar();
}
```

Explicação

- O `switch(modo)` pula direto para o `case` do modo atual. O `break` encerra o `case`; sem ele, o código continuaria executando o `case` seguinte.
- O `default` cobre qualquer valor não previsto. É uma boa prática, mesmo que "nunca deva acontecer".
- `aplicarModo()` começa com `desativarPiscar()` em todos os LEDs. Sem isso, ao sair do modo 2 para o 3, o amarelo e o verde continuariam piscando.
- O ternário `(modo == TOTAL_MODOS - 1) ? 0 : modo + 1` faz o modo "dar a volta": depois do último, volta ao 0.
- `aplicarModo()` só é chamada quando o modo muda. O que precisa rodar sempre (o `atualizar()` dos LEDs, que faz o pisca acontecer) fica no `loop()`.

Gabarito 4 — Barra de nível

```
// Exercício 4 — Barra de nível no LCD com for

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

const uint8_t NIVEL_MAXIMO = 10;
uint8_t nivel = 0;
bool precisaAtualizarTela = true;

void subirNivel()
{
    if(nivel == NIVEL_MAXIMO) return;   // cláusula de guarda
    nivel++;
    precisaAtualizarTela = true;
}

void descerNivel()
{
    if(nivel == 0) return;              // cláusula de guarda
    nivel--;
    precisaAtualizarTela = true;
}

void atualizarLeds()
{
    ledVerde.setEstadoLed(nivel >= 1);
    ledAmarelo.setEstadoLed(nivel >= 4);

    if(nivel == NIVEL_MAXIMO)
    {
        ledVermelho.ativarPiscar(200);
    }
    else if(nivel >= 8)
    {
        ledVermelho.desativarPiscar();
```

```
        ledVermelho.ligar();
    }
    else
    {
        ledVermelho.desativarPiscar();
    }
}

void mostrarNivel()
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Nivel: ");
    lcd.print(nivel * 10);
    lcd.print("%");

    lcd.setCursor(0, 1);
    lcd.print("[");

    for(uint8_t i = 0; i < nivel; i++)
    {
        lcd.print("#");
    }

    for(uint8_t i = nivel; i < NIVEL_MAXIMO; i++)
    {
        lcd.print("-");
    }

    lcd.print("]");
}

void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    botaoB.iniciar();
}

void loop()
{
    botaoA.atualizar();
```

```
    botaoB.atualizar();

    if(botaoA.pressionou()) subirNivel();
    if(botaoB.pressionou()) descerNivel();

    ledVermelho.atualizar();
    ledAmarelo.atualizar();
    ledVerde.atualizar();

    if(!precisaAtualizarTela) return;

    precisaAtualizarTela = false;
    atualizarLeds();
    mostrarNivel();
}
```

Explicação

- As cláusulas de guarda em `subirNivel()` e `descerNivel()` resolvem os limites logo na primeira linha: se já está no limite, a função termina e nada muda.
- O primeiro `for` imprime um `#` para cada nível; o segundo começa onde o primeiro parou e completa com `-` até 10. Juntos, sempre desenham 10 caracteres.
- Nos LEDs, `nivel >= 1` já é um valor `true/false`, que vai direto para `setEstadoLed()`.
- O `if / else if / else` do vermelho é testado de cima para baixo: o caso mais específico (nível 10) vem primeiro. Se `nivel >= 8` viesse antes, o nível 10 nunca chegaria a piscar.

Gabarito 5 — Cronômetro

```
// Exercício 5 — Cronômetro com millis()

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

const uint32_t INTERVALO_TELA_ms = 100;
```

```
bool rodando = false;
uint32_t inicio_ms = 0;        // quando o cronômetro foi (re)iniciado
uint32_t acumulado_ms = 0;     // tempo guardado das corridas anteriores
uint32_t ultimaTela_ms = 0;

uint32_t tempoTotal_ms()
{
    return rodando ? acumulado_ms + (millis() - inicio_ms) : acumulado_ms;
}

void iniciarOuPausar()
{
    if(rodando)
    {
        acumulado_ms += millis() - inicio_ms;
        rodando = false;
    }
    else
    {
        inicio_ms = millis();
        rodando = true;
    }
}

void zerar()
{
    if(rodando) return;   // cláusula de guarda: só zera pausado
    acumulado_ms = 0;
}

void mostrarTempo(uint32_t total_ms)
{
    const uint32_t minutos  = total_ms / 60000;
    const uint32_t segundos = (total_ms / 1000) % 60;
    const uint32_t decimos  = (total_ms / 100) % 10;

    lcd.setCursor(0, 0);
    lcd.print(rodando ? "Rodando " : "Pausado ");

    lcd.setCursor(0, 1);
    if(minutos < 10) lcd.print("0");
    lcd.print(minutos);
    lcd.print(":");
    if(segundos < 10) lcd.print("0");
    lcd.print(segundos);
```

```
    lcd.print(".");
    lcd.print(decimos);
}

void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    botaoB.iniciar();
}

void loop()
{
    botaoA.atualizar();
    botaoB.atualizar();

    if(botaoA.pressionou()) iniciarOuPausar();
    if(botaoB.pressionou()) zerar();

    ledVerde.setEstadoLed(rodando);
    ledAmarelo.setEstadoLed(!rodando && acumulado_ms > 0);
    ledVerde.atualizar();
    ledAmarelo.atualizar();

    if(millis() - ultimaTela_ms < INTERVALO_TELA_ms) return;

    ultimaTela_ms = millis();
    mostrarTempo(tempoTotal_ms());
}
```

Explicação

- Duas variáveis guardam o tempo: `acumulado_ms` (o que já foi contado antes das pausas) e `inicio_ms` (quando voltou a rodar). Ao pausar, o trecho atual é somado ao acumulado.
- `tempoTotal_ms()` usa o ternário: rodando, soma o trecho atual; pausado, devolve só o acumulado.
- A conversão usa divisão inteira e resto: 1 minuto = 60000 ms, `(total / 1000) % 60` são os segundos que sobram do minuto e `(total / 100) % 10` é o décimo de segundo.
- A tela é atualizada a cada 100 ms com a mesma lógica do exercício 1, mas em forma de cláusula de guarda: "ainda não passou o tempo? então `return`".

- `"Rodando "` e `"Pausado "` têm o mesmo tamanho, e o tempo sempre ocupa as mesmas colunas. Por isso não é preciso `lcd.clear()`, e a tela não pisca.

Gabarito 11 — Monitor de temperatura com DHT22

```
// Exercício 11 — Monitor de temperatura com DHT22 (ESP32 no Wokwi)

#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

const uint8_t PINO_DHT = 4;
const uint32_t INTERVALO_LEITURA_ms = 5000;
const int16_t ALERTA_DECIMOS = 300;   // 30,0 °C

LiquidCrystal_I2C lcd(0x27, 16, 2);   // endereço 0x27, 16 colunas, 2 linhas
DHT dht(PINO_DHT, DHT22);

Led ledVermelho(25);
Led ledVerde(27);

uint32_t ultimaLeitura_ms = 0;
int16_t ultimaExibida_decimos = -9999;   // valor impossível: força a
primeira exibição

void mostrarTemperatura(int16_t temperatura_decimos)
{
    const bool alerta = temperatura_decimos >= ALERTA_DECIMOS;

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(temperatura_decimos / 10.0, 1);
    lcd.print((char)223);   // símbolo de grau no LCD
    lcd.print("C");

    lcd.setCursor(0, 1);
    lcd.print(alerta ? "ALERTA: quente!" : "Normal");

    alerta
    ? ledVermelho.ativarPiscar(300)
    : ledVermelho.desativarPiscar();

    ledVerde.setEstadoLed(!alerta);
}
```

```
void setup()
{
    lcd.init();
    lcd.backlight();
    dht.begin();
    ledVermelho.iniciar();
    ledVerde.iniciar();

    lcd.print("Lendo sensor...");
}

void loop()
{
    ledVermelho.atualizar();
    ledVerde.atualizar();

    // Guarda 1: ainda não passaram 5 s desde a última leitura
    if(millis() - ultimaLeitura_ms < INTERVALO_LEITURA_ms) return;

    ultimaLeitura_ms = millis();
    const float temperatura = dht.readTemperature();

    // Guarda 2: a leitura falhou
    if(isnan(temperatura))
    {
        lcd.clear();
        lcd.print("Erro no sensor");
        ultimaExibida_decimos = -9999;   // força exibir quando voltar
        return;
    }

    const int16_t temperatura_decimos = round(temperatura * 10);

    // Guarda 3: o valor não mudou, não há o que atualizar
    if(temperatura_decimos == ultimaExibida_decimos) return;

    ultimaExibida_decimos = temperatura_decimos;
    mostrarTemperatura(temperatura_decimos);
}
```

Explicação

- O `loop()` segue o fluxograma com cláusulas de guarda: são três "portas" seguidas. Só quando passa pelas três é que o LCD é reescrito.

- **Guarda 1 (tempo):** mesma técnica do exercício 1. Enquanto não completar 5 s, a função sai logo depois de atualizar os LEDs. Por isso o pisca do LED vermelho continua funcionando entre uma leitura e outra.
- **Guarda 2 (falha):** se o sensor não responder, `readTemperature()` devolve NaN. O programa avisa e coloca `-9999` na última temperatura mostrada, para que a próxima leitura válida apareça mesmo que seja igual à anterior.
- **Guarda 3 (variação):** a temperatura é convertida em décimos de grau inteiros (25,3 °C vira 253). Comparar inteiros é seguro; comparar `float` com `==` pode falhar por diferenças minúsculas que nem aparecem no LCD.
- O valor inicial `-9999` é impossível para o DHT22 (mínimo de −40 °C, ou −400 décimos). Assim a primeira leitura sempre é considerada uma "variação" e aparece na tela.
- `temperatura_decimos / 10.0` volta a ser um número com vírgula para exibir. O `10.0` (e não `10`) é importante: dividir dois inteiros descartaria a parte decimal.
- `alerta` já é `true/false`, então serve direto para o ternário do texto, para escolher entre `ativarPiscar` e `desativarPiscar` e para o `setEstadoLed` do LED verde.
- **Para testar no Wokwi:** inicie a simulação, clique no DHT22 e arraste a temperatura. A tela só muda na próxima leitura (até 5 s depois) e só se o valor for diferente. Passe de 30 °C para ver o alerta.

Apêndice — Classes Led e Botao (arquivo único para o Tinkercad)

São as mesmas classes feitas em aula, com o `.h` e o `.cpp` de cada uma reunidos em um bloco só, porque o Tinkercad aceita apenas um arquivo. Cole este bloco no topo do código, no lugar da linha `// >>> Cole aqui o bloco do Apêndice <<<`. A biblioteca do LCD fica no início de cada exercício.

```
// ===== Classes Led e Botao (versão arquivo único para o Tinkercad) =====
// (a biblioteca do LCD é incluída em cada exercício)

// ---------- Classe Led ----------
class Led
{
  private:
    uint8_t _pinoLed;
    bool _estadoLed = 0;
    bool _estaPiscando = false;
    uint32_t _tempoAcaoAnterior_ms = 0;
    uint32_t _tempoEsperaAlternar_ms = 0;

  public:
    Led(uint8_t pino);
```

```
    void iniciar();
    void atualizar();
    void ligar();
    void desligar();
    void ativarPiscar(uint32_t tempoEspera_ms = 500);
    void desativarPiscar();
    void alternar();

    uint8_t getPinoLed();
    void setEstadoLed(bool estado);
};

Led::Led(uint8_t pino) : _pinoLed(pino)
{
}

void Led::iniciar()
{
    pinMode(_pinoLed, OUTPUT);
    digitalWrite(_pinoLed, _estadoLed);
    _tempoAcaoAnterior_ms = millis();
}

void Led::atualizar()
{
    if(_estaPiscando)
    {
        const uint32_t tempoDecorrido = millis() - _tempoAcaoAnterior_ms;

        if(tempoDecorrido >= _tempoEsperaAlternar_ms)
        {
            _tempoAcaoAnterior_ms = millis();
            alternar();
        }
    }

    digitalWrite(_pinoLed, _estadoLed);
}

void Led::ligar()
{
    _estadoLed = HIGH;
}

void Led::desligar()
```

```
{
    _estadoLed = LOW;
}

void Led::ativarPiscar(uint32_t tempoEspera_ms)
{
    _estaPiscando = true;
    _tempoEsperaAlternar_ms = tempoEspera_ms;
}

void Led::desativarPiscar()
{
    _estaPiscando = false;
    _estadoLed = LOW;
}

void Led::alternar()
{
    _estadoLed = !_estadoLed;
}

uint8_t Led::getPinoLed()
{
    return _pinoLed;
}

void Led::setEstadoLed(bool estado)
{
    _estadoLed = estado;
}

// ---------- Classe Botao ----------
class Botao
{
    private:
        uint8_t _pinoBotao;
        bool _estadoAtualBotao = HIGH;
        bool _estadoAnteriorBotao = HIGH;
        bool _pressionou = false;
        bool _soltou = false;
        bool _estadoUltimaAcao = HIGH;

        uint32_t _ultimaMudanca_ms = 0;
        uint32_t _tempoDebounce_ms = 20;

        uint32_t tempoDecorrido();
```

```
    public:
        Botao(uint8_t pino);

        void iniciar();
        void atualizar();
        bool pressionou();
        bool soltou();
        void setTempoDebounce(uint32_t tempo_ms);
};

Botao::Botao(uint8_t pino) : _pinoBotao(pino)
{
}

void Botao::iniciar()
{
    pinMode(_pinoBotao, INPUT_PULLUP);
}

void Botao::atualizar()
{
    _pressionou = false;
    _soltou = false;

    _estadoAtualBotao = digitalRead(_pinoBotao);

    if(_estadoAtualBotao != _estadoAnteriorBotao)
    {
        _ultimaMudanca_ms = millis();
        _estadoAnteriorBotao = _estadoAtualBotao;
        return;
    }

    if(tempoDecorrido() < _tempoDebounce_ms) return;

    if(_estadoUltimaAcao == _estadoAtualBotao) return;

    _estadoUltimaAcao = _estadoAtualBotao;

    _estadoAtualBotao
    ? _soltou = true
    : _pressionou = true;
}

bool Botao::pressionou()
```

```
{
    return _pressionou;
}

bool Botao::soltou()
{
    return _soltou;
}

uint32_t Botao::tempoDecorrido()
{
    return millis() - _ultimaMudanca_ms;
}

void Botao::setTempoDebounce(uint32_t tempo_ms)
{
    _tempoDebounce_ms = tempo_ms;
}
// ===== Fim das classes =====
```

Gabarito 6 — Teste de reflexo

```
// Exercício 6 — Teste de reflexo (máquina de estados com switch)

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

const uint8_t ESTADO_INICIO     = 0;
const uint8_t ESTADO_AGUARDANDO = 1;
const uint8_t ESTADO_MEDINDO    = 2;
const uint8_t ESTADO_RESULTADO  = 3;

uint8_t estado = ESTADO_INICIO;

uint32_t marcaTempo_ms = 0;
uint32_t esperaSorteada_ms = 0;

void mostrarTela(const char* linha1, const char* linha2)
```

```
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(linha1);
    lcd.setCursor(0, 1);
    lcd.print(linha2);
}

void irParaInicio()
{
    ledVermelho.desativarPiscar();
    ledVerde.desligar();
    mostrarTela("Teste de reflexo", "A: comecar");
    estado = ESTADO_INICIO;
}

void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    botaoB.iniciar();
    irParaInicio();
}

void loop()
{
    botaoA.atualizar();
    botaoB.atualizar();

    switch(estado)
    {
        case ESTADO_INICIO:
            if(botaoA.pressionou())
            {
                randomSeed(millis());
                esperaSorteada_ms = random(2000, 5001);
                marcaTempo_ms = millis();
                mostrarTela("Prepare-se...", "Aperte B no LED");
                estado = ESTADO_AGUARDANDO;
            }
            break;

        case ESTADO_AGUARDANDO:
```

```
            if(botaoB.pressionou())
            {
                ledVermelho.ativarPiscar(150);
                mostrarTela("Queimou largada!", "A: tentar de novo");
                estado = ESTADO_RESULTADO;
            }
            else if(millis() - marcaTempo_ms >= esperaSorteada_ms)
            {
                ledVerde.ligar();
                marcaTempo_ms = millis();
                estado = ESTADO_MEDINDO;
            }
            break;

        case ESTADO_MEDINDO:
            if(botaoB.pressionou())
            {
                const uint32_t reacao_ms = millis() - marcaTempo_ms;
                ledVerde.desligar();

                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Reacao: ");
                lcd.print(reacao_ms);
                lcd.print(" ms");

                lcd.setCursor(0, 1);
                if(reacao_ms < 250)      lcd.print("Excelente!");
                else if(reacao_ms < 400) lcd.print("Bom!");
                else                     lcd.print("Treine mais!");

                estado = ESTADO_RESULTADO;
            }
            break;

        case ESTADO_RESULTADO:
            if(botaoA.pressionou()) irParaInicio();
            break;
    }

    ledVermelho.atualizar();
    ledVerde.atualizar();
}
```

Explicação

- A variável `estado` diz em que etapa o jogo está. Dar nomes aos números (`ESTADO_INICIO`, `ESTADO_MEDINDO`...) torna o `switch` fácil de ler.
- Cada `case` só faz duas coisas: espera um acontecimento (um botão ou um tempo) e, quando ele ocorre, prepara a próxima etapa e troca o `estado`.
- `marcaTempo_ms` é reaproveitada: primeiro marca o início da espera; depois marca o momento em que o LED acendeu. A reação é `millis() - marcaTempo_ms`.
- `random(2000, 5001)` sorteia de 2000 a 5000 (o valor máximo não entra no sorteio). `randomSeed(millis())` usa o instante do clique como "semente", que muda a cada partida.
- No `ESTADO_AGUARDANDO`, o `if` do botão B vem antes do `else if` do tempo: apertar antes do sinal é tratado primeiro.
- O LCD só é escrito nas mudanças de estado, e não a cada volta do loop.

Gabarito 7 — Semáforo com botão de pedestre

```
// Exercício 7 — Semáforo com botão de pedestre

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);

const uint8_t VERDE    = 0;
const uint8_t AMARELO  = 1;
const uint8_t VERMELHO = 2;

const uint32_t DURACAO_VERDE_ms    = 5000;
const uint32_t DURACAO_AMARELO_ms  = 2000;
const uint32_t DURACAO_VERMELHO_ms = 5000;
const uint32_t VERDE_MINIMO_ms     = 2000;
const uint32_t INTERVALO_TELA_ms   = 250;

uint8_t estado = VERDE;
uint32_t inicioEstado_ms = 0;
uint32_t ultimaTela_ms = 0;
bool pedidoPedestre = false;
```

```
uint32_t duracaoDoEstado()
{
    switch(estado)
    {
        case VERDE:   return DURACAO_VERDE_ms;
        case AMARELO: return DURACAO_AMARELO_ms;
        default:      return DURACAO_VERMELHO_ms;
    }
}

void mudarEstado(uint8_t novoEstado)
{
    estado = novoEstado;
    inicioEstado_ms = millis();

    if(estado == VERMELHO) pedidoPedestre = false;

    ledVerde.setEstadoLed(estado == VERDE);
    ledAmarelo.setEstadoLed(estado == AMARELO);
    ledVermelho.setEstadoLed(estado == VERMELHO);

    lcd.clear();
    lcd.setCursor(0, 0);
    switch(estado)
    {
        case VERDE:    lcd.print("VERDE: siga"); break;
        case AMARELO:  lcd.print("AMARELO: atencao"); break;
        case VERMELHO: lcd.print("VERMELHO: pare"); break;
    }
}

void mostrarContagem()
{
    const uint32_t decorrido_ms = millis() - inicioEstado_ms;
    const uint32_t duracao_ms = duracaoDoEstado();
    const uint32_t restante_s = (decorrido_ms >= duracao_ms) ? 0 :
(duracao_ms - decorrido_ms + 999) / 1000;

    lcd.setCursor(0, 1);
    lcd.print("Restam: ");
    lcd.print(restante_s);
    lcd.print("s ");
    lcd.print(pedidoPedestre ? "PED" : "   ");
}

void setup()
```

```
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    mudarEstado(VERDE);
}

void loop()
{
    botaoA.atualizar();

    if(botaoA.pressionou() && estado == VERDE) pedidoPedestre = true;

    const uint32_t decorrido_ms = millis() - inicioEstado_ms;

    switch(estado)
    {
        case VERDE:
            if(decorrido_ms >= DURACAO_VERDE_ms) mudarEstado(AMARELO);
            else if(pedidoPedestre && decorrido_ms >= VERDE_MINIMO_ms)
mudarEstado(AMARELO);
            break;

        case AMARELO:
            if(decorrido_ms >= DURACAO_AMARELO_ms) mudarEstado(VERMELHO);
            break;

        case VERMELHO:
            if(decorrido_ms >= DURACAO_VERMELHO_ms) mudarEstado(VERDE);
            break;
    }

    ledVermelho.atualizar();
    ledAmarelo.atualizar();
    ledVerde.atualizar();

    if(millis() - ultimaTela_ms < INTERVALO_TELA_ms) return;

    ultimaTela_ms = millis();
    mostrarContagem();
}
```

Explicação

- `mudarEstado()` concentra tudo o que acontece numa troca: guarda o estado, anota o horário, acerta os 3 LEDs (`estado == VERDE` já é `true/false`) e escreve o nome no LCD.
- No `loop()`, cada `case` só compara o tempo decorrido com a duração do seu estado. Fica fácil mudar os tempos: basta alterar as constantes.
- O pedestre não muda o estado na hora: ele só liga `pedidoPedestre`. Quem decide é o `case VERDE`, que exige pelo menos 2 s de verde. O pedido é apagado quando o sinal fecha (vermelho).
- Em `duracaoDoEstado()`, cada `case` usa `return`, que já sai da função; por isso não precisa de `break`.
- O `+ 999` na contagem regressiva arredonda para cima: com 4,2 s restantes, o LCD mostra 5 s, e não 4 s.

Gabarito 8 — Clique curto ou clique longo?

```
// Exercício 8 — Clique curto x clique longo

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);

const uint32_t TEMPO_CLIQUE_LONGO_ms = 800;

bool segurando = false;
bool vermelhoPiscando = false;
uint32_t inicioPressao_ms = 0;

void mostrarAcao(const char* acao, uint32_t duracao_ms)
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(acao);
    lcd.setCursor(0, 1);
    lcd.print("Durou ");
    lcd.print(duracao_ms);
    lcd.print(" ms");
```

```
}

void tratarBotaoA()
{
    if(botaoA.pressionou())
    {
        segurando = true;
        inicioPressao_ms = millis();
        return;
    }

    if(!botaoA.soltou()) return;   // nada aconteceu com o botão

    segurando = false;
    const uint32_t duracao_ms = millis() - inicioPressao_ms;

    if(duracao_ms < TEMPO_CLIQUE_LONGO_ms)
    {
        ledAmarelo.alternar();
        mostrarAcao("Clique CURTO", duracao_ms);
        return;
    }

    vermelhoPiscando = !vermelhoPiscando;

    vermelhoPiscando
    ? ledVermelho.ativarPiscar(300)
    : ledVermelho.desativarPiscar();

    mostrarAcao("Clique LONGO", duracao_ms);
}

void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    lcd.print("Clique ou segure");
}

void loop()
{
    botaoA.atualizar();
```

```
    tratarBotaoA();

    // Verde aceso enquanto o botão está pressionado
    ledVerde.setEstadoLed(segurando);

    ledVermelho.atualizar();
    ledAmarelo.atualizar();
    ledVerde.atualizar();
}
```

Explicação

- `tratarBotaoA()` é escrita com cláusulas de guarda, como no fluxograma do Classroom: cada `return` encerra a função assim que não há mais nada a fazer, e o código não vira uma "escada" de `if` dentro de `if`.
- Ao pressionar, guardamos o `millis()`. Ao soltar, `millis() - inicioPressao_ms` é quanto tempo o botão ficou apertado.
- Clique curto: `alternar()` inverte o LED amarelo e a função termina com `return`. Se chegou à parte de baixo da função, o clique só pode ter sido longo, e não é preciso um `else`.
- O ternário com duas chamadas de método (`ativarPiscar` ou `desativarPiscar`) é o mesmo estilo usado dentro de `Botao::atualizar()` para escolher entre `_soltou` e `_pressionou`.
- `segurando` acompanha o botão de verdade e vai direto para o LED verde.

Gabarito 9 — Crie a sua classe: Contador

```
// Exercício 9 — Criando a sua própria classe: Contador

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

// ---------- Classe Contador ----------
class Contador
{
    private:
        uint16_t _valor = 0;
        uint16_t _limite;

    public:
        Contador(uint16_t limite);

        void incrementar();
        void decrementar();
        void zerar();
```

```
        uint16_t getValor();
        uint16_t getLimite();
        bool atingiuLimite();
        bool estaZerado();
};

Contador::Contador(uint16_t limite) : _limite(limite)
{
}

void Contador::incrementar()
{
    if(_valor >= _limite) return;
    _valor++;
}

void Contador::decrementar()
{
    if(_valor == 0) return;
    _valor--;
}

void Contador::zerar()
{
    _valor = 0;
}

uint16_t Contador::getValor()
{
    return _valor;
}

uint16_t Contador::getLimite()
{
    return _limite;
}

bool Contador::atingiuLimite()
{
    return _valor >= _limite;
}

bool Contador::estaZerado()
{
    return _valor == 0;
}
```

```
// ---------- Programa principal ----------
#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

Contador vagas(5);   // ex.: estacionamento com 5 vagas
bool precisaAtualizar = true;

void atualizarLedsETela()
{
    ledVermelho.desativarPiscar();
    ledAmarelo.desligar();
    ledVerde.desligar();

    if(vagas.atingiuLimite())
    {
        ledVermelho.ativarPiscar(300);
    }
    else if(vagas.estaZerado())
    {
        ledVerde.ligar();
    }
    else
    {
        ledAmarelo.ligar();
    }

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Ocupadas: ");
    lcd.print(vagas.getValor());
    lcd.print("/");
    lcd.print(vagas.getLimite());

    lcd.setCursor(0, 1);
    lcd.print(vagas.atingiuLimite() ? "LOTADO" : "Ha vagas");
}

void setup()
```

```
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    botaoB.iniciar();
}

void loop()
{
    botaoA.atualizar();
    botaoB.atualizar();

    if(botaoA.pressionou())
    {
        vagas.incrementar();
        precisaAtualizar = true;
    }

    if(botaoB.pressionou())
    {
        vagas.decrementar();
        precisaAtualizar = true;
    }

    if(precisaAtualizar)
    {
        precisaAtualizar = false;
        atualizarLedsETela();
    }

    ledVermelho.atualizar();
    ledAmarelo.atualizar();
    ledVerde.atualizar();
}
```

Explicação

- A classe segue o mesmo modelo de `Led` e `Botao`: atributos em `private` com `_` no início, métodos em `public`, declaração da classe primeiro e implementação de cada método com `Contador::` depois.
- O construtor usa a lista de inicialização (`: _limite(limite)`), igual ao `Led(uint8_t pino) : _pinoLed(pino)`.

- Os atributos são privados: o `loop()` não consegue escrever `vagas._valor = 50;`. A única forma de mudar o valor é pelos métodos, e eles garantem os limites com cláusulas de guarda.
- Os métodos `getValor()` e `getLimite()` só leem os atributos. `atingiuLimite()` e `estaZerado()` devolvem `true/false`, o que deixa os `if` do programa principal legíveis quase como uma frase.
- O mesmo `Contador` serviria para outras situações (pessoas numa sala, peças numa esteira) trocando apenas o limite no construtor.

Gabarito 10 — Desafio integrador: temporizador de cozinha

```
// Exercício 10 — Desafio integrador: temporizador de cozinha

// >>> Cole aqui o bloco do Apêndice (classes Led e Botao) <<<

#include <Adafruit_LiquidCrystal.h>
Adafruit_LiquidCrystal lcd(0);   // LCD 16x2 I2C (endereço 0 no Tinkercad)

Led ledVermelho(10);
Led ledAmarelo(9);
Led ledVerde(8);

Botao botaoA(7);
Botao botaoB(6);

const uint8_t AJUSTE   = 0;
const uint8_t CONTANDO = 1;
const uint8_t ALARME   = 2;

const uint32_t PASSO_AJUSTE_ms    = 10000;   // +10 s por clique
const uint32_t TEMPO_MAXIMO_ms    = 300000;  // 5 minutos
const uint32_t INTERVALO_TELA_ms  = 200;
const uint8_t  LARGURA_BARRA      = 16;

uint8_t estado = AJUSTE;
uint32_t tempoProgramado_ms = 0;
uint32_t inicioContagem_ms = 0;
uint32_t ultimaTela_ms = 0;

void imprimirTempo(uint32_t tempo_ms)
{
    const uint32_t totalSegundos = (tempo_ms + 999) / 1000;   // arredonda
para cima
    const uint32_t minutos = totalSegundos / 60;
```

```
    const uint32_t segundos = totalSegundos % 60;

    if(minutos < 10) lcd.print("0");
    lcd.print(minutos);
    lcd.print(":");
    if(segundos < 10) lcd.print("0");
    lcd.print(segundos);
}

void mostrarAjuste()
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Ajuste: ");
    imprimirTempo(tempoProgramado_ms);
    lcd.setCursor(0, 1);
    lcd.print("A:+10s  B:inicia");
}

void irParaAjuste()
{
    ledVermelho.desativarPiscar();
    ledAmarelo.desativarPiscar();
    ledVerde.desativarPiscar();
    ledAmarelo.ligar();
    estado = AJUSTE;
    mostrarAjuste();
}

void irParaContagem()
{
    if(tempoProgramado_ms == 0) return;   // nada para contar

    ledAmarelo.desligar();
    ledVerde.ativarPiscar(1000);
    inicioContagem_ms = millis();
    ultimaTela_ms = 0;
    estado = CONTANDO;
}

void irParaAlarme()
{
    ledVerde.desativarPiscar();
    ledVermelho.ativarPiscar(200);
    ledAmarelo.ativarPiscar(200);
    ledVerde.ativarPiscar(200);
```

```
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("TEMPO ESGOTADO!");
    lcd.setCursor(0, 1);
    lcd.print("A ou B: parar");
    estado = ALARME;
}

void mostrarContagem(uint32_t restante_ms)
{
    lcd.setCursor(0, 0);
    lcd.print("Faltam: ");
    imprimirTempo(restante_ms);
    lcd.print("   ");

    // Barra de progresso: quanto já passou
    const uint32_t passado_ms = tempoProgramado_ms - restante_ms;
    const uint8_t preenchidos = (passado_ms * LARGURA_BARRA) /
tempoProgramado_ms;

    lcd.setCursor(0, 1);
    for(uint8_t i = 0; i < LARGURA_BARRA; i++)
    {
        lcd.print(i < preenchidos ? "#" : "-");
    }
}

void setup()
{
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    ledVermelho.iniciar();
    ledAmarelo.iniciar();
    ledVerde.iniciar();
    botaoA.iniciar();
    botaoB.iniciar();
    irParaAjuste();
}

void loop()
{
    botaoA.atualizar();
    botaoB.atualizar();

    switch(estado)
```

```
    {
        case AJUSTE:
            if(botaoA.pressionou())
            {
                tempoProgramado_ms += PASSO_AJUSTE_ms;
                if(tempoProgramado_ms > TEMPO_MAXIMO_ms) tempoProgramado_ms =
PASSO_AJUSTE_ms;
                mostrarAjuste();
            }
            if(botaoB.pressionou()) irParaContagem();
            break;

        case CONTANDO:
        {
            if(botaoB.pressionou())
            {
                irParaAjuste();   // cancela e volta ao ajuste
                break;
            }

            const uint32_t passado_ms = millis() - inicioContagem_ms;

            if(passado_ms >= tempoProgramado_ms)
            {
                irParaAlarme();
                break;
            }

            if(millis() - ultimaTela_ms >= INTERVALO_TELA_ms)
            {
                ultimaTela_ms = millis();
                mostrarContagem(tempoProgramado_ms - passado_ms);
            }
            break;
        }

        case ALARME:
            if(botaoA.pressionou() || botaoB.pressionou()) irParaAjuste();
            break;
    }

    ledVermelho.atualizar();
    ledAmarelo.atualizar();
    ledVerde.atualizar();
}
```

Explicação

- O programa é uma máquina de estados com três etapas. Cada mudança tem a sua função (`irParaAjuste` , `irParaContagem` , `irParaAlarme`), que deixa LEDs e tela prontos para a etapa nova. Assim o `switch` só decide **quando** trocar.
- `irParaContagem()` começa com uma cláusula de guarda: com o tempo em zero, não há o que contar e nada muda.
- Dentro de `case CONTANDO` , o código está entre `{ }` porque declara uma variável (`passado_ms`) dentro do `case` . Os `break` internos funcionam como cláusulas de guarda: cancelou ou acabou o tempo, sai do `switch` .
- A barra de progresso é uma regra de três: `passado × 16 ÷ total` diz quantos dos 16 caracteres devem estar preenchidos. O `for` percorre as 16 posições e o ternário escolhe `#` ou `-` em cada uma.
- Nenhum `delay()` foi usado: mesmo durante a contagem, os botões respondem na hora e os LEDs continuam piscando.

Para ir além: adicione uma pausa na contagem usando a mesma técnica do cronômetro do exercício 5.